

Fundamental CS for AI Era

Building human-centered digital products in the AI era



I'm Zain Fathoni

Front-end developer based in Yogyakarta, Indonesia. Previously backend, manager, frontend, now fullstack.

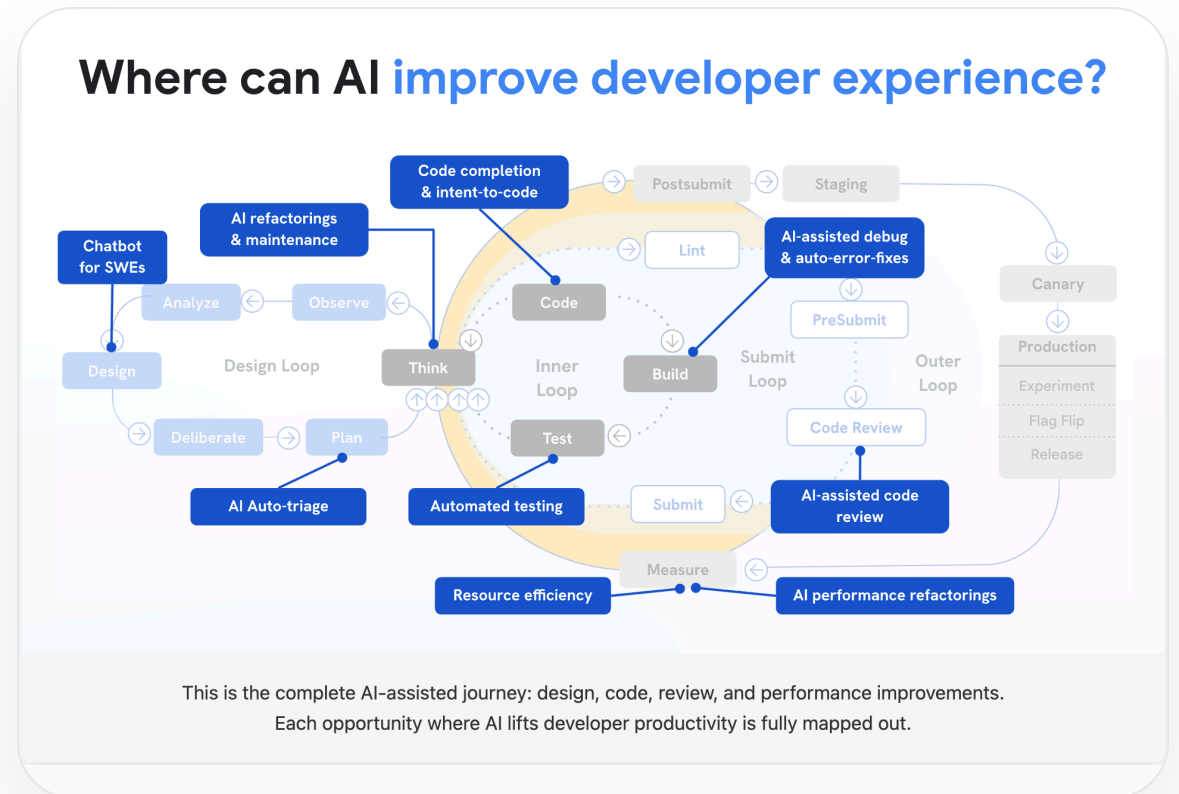
Community builder · AI-assisted engineering practitioner · Mentor Spotlight at BUILD Camp.

<https://zainf.dev>

AI makes code generation fast and accessible

Anyone in this room can ship working code today — developer or not.

Generating code is no longer the hard part.
We all agree on that.



Understanding becomes the bottleneck

AI generates code faster than we can understand it.

The constraint moves from **writing** code to **understanding what changed, why it works, and what it might break.**

Geoffrey Litt · 2 July 2026

Hot take: I think it's still important to understand the code that our agents write!

Understanding matters not just to verify, but to participate. — “Understanding is the new bottleneck”

Imagine calm confidence in any codebase

Opening an unfamiliar repository without dread.

Reviewing an AI-generated PR and **knowing** what to look at first.

That feeling is trainable — and AI itself can train it.

AI
×
CS

The win is not “skip fundamentals.” The win is “learn fundamentals faster, with feedback.”

“AI means I can skip fundamentals”

Tempting — but this is what it looks like from the receiving end:

Dex Horthy · 20 June 2026

If people are tokenmaxxing bugs into production with kLOC PRs that they didn't read themselves, those people shouldn't have jobs.

“Coaching juniors on SWE fundamentals is hard work and it takes time.”

Fundamentals are the AI multiplier

Data structures, algorithms, systems thinking, debugging, trade-offs.

They give you the **taste** to judge AI output and the **words** to steer it.

Gergely Orosz · 2 July 2026

If you don't know what good code looks like, you will have no idea if what the models generate are any good.

"This is exactly why experienced software engineers are valuable and will be valuable." (via @mitchellh)

P7 · WE CAN DO THIS

A loop anyone can run

Not a talent. A habit — five moves, repeated on every change:

 **Read** — pick which code is worth reading



 **Model** — form a theory of how it works



 **Ask** — probe the theory with AI



 **Verify** — tests, types, experiments



 **Explain** — in your own words

Your first doable step: **/teach**

An open-source Claude Code skill that turns that loop into a **persistent learning workspace**.

1. Copy `skills/teach` from github.com/zainfathoni/agent-workflows into `~/.claude/skills`
2. Run `/teach` with any topic you want to learn
3. Answer the mission interview, take one lesson and its quiz
4. Return anytime — it remembers where you left off

 **MISSION.md** — why you're learning

 **lessons/** — short, single-win lessons

 **quizzes** — speed regulators for real understanding

 **learning-records/** — evidenced progress

Quizzes as speed regulators — the same technique Geoffrey Litt proposes for understanding AI code.

You can learn anything you want

Not just CS fundamentals. The same workspace teaches you:

- a new framework or language
- an unfamiliar codebase, one PR at a time
- algorithms and system design
- even non-code topics

Instead of:

passive tutorials you abandon by chapter three.

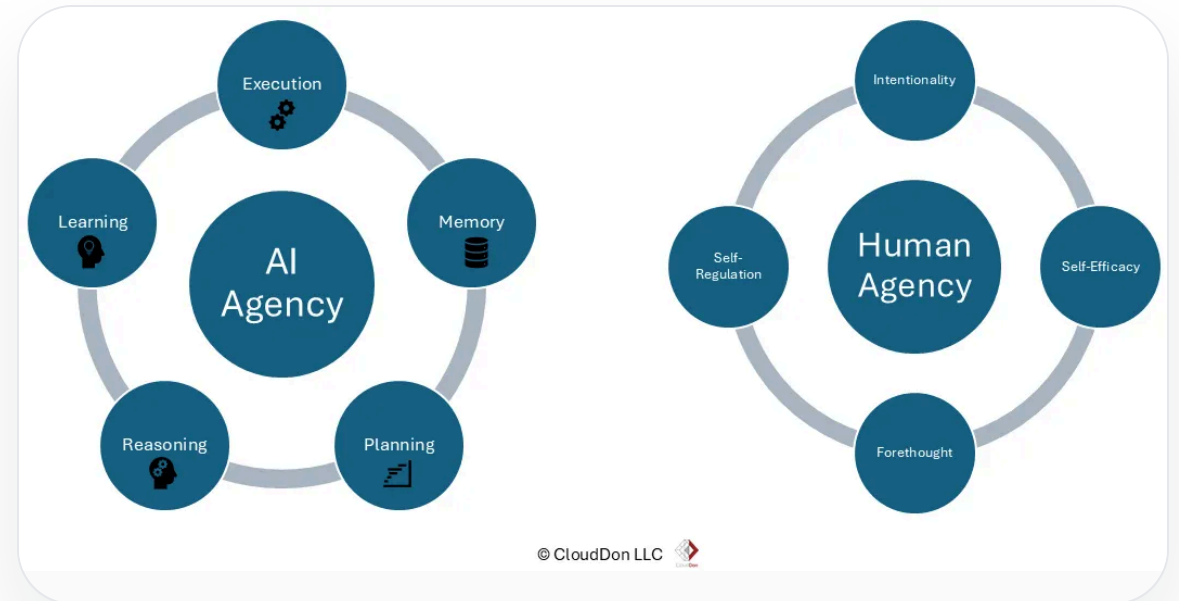
You get:

a personal curriculum with lessons, quizzes, and memory — paced by what you actually retain.

Engineers become better problem framers

In the AI era, human-centered builders are not the people who type the most code.

They are the people who can **frame problems clearly enough for humans and machines to solve together.**



Live Demo

Let's learn something together — for real, for the next hour.



You pick the topic. You answer the quizzes.

How the next hour works

1. **Pick** (10 min) — you shout topics, we vote on one
2. **Mission** (15 min) — `/teach` interviews us; you answer, I type
3. **Learn** (20 min) — we take the first generated lesson together
4. **Quiz** (15 min) — you answer; wrong answers steer the next lesson

Ground rule:

I will not edit the AI's questions or lessons. What you see is what the skill does.

Fallback topic:

"How does the code AI just wrote for me actually work?" — the theme of this talk.

What we'll watch it build

A learning workspace, growing live on screen:

- `MISSION.md` — the room's shared goal
- `RESOURCES.md` — sources we trust
- `lessons/*.html` — one win per lesson
- `learning-records/*.md` — proof of what we learned

Understanding matters not just to verify, but to participate.

Geoffrey Litt — this hour is that idea, practiced.

References

- Geoffrey Litt — [Understanding is the new bottleneck](#)
- Dex Horthy — [against lazy engineering](#)
- Gergely Orosz — [knowing what good code looks like](#)
- Zain Fathoni — [A to Z #3: my personal experience](#)
- The `/teach` skill — github.com/zainfathoni/agent-workflows
- Dan Roam — [The Pop-up Pitch](#)

Fundamentals are not the past.

They are how we steer the future.

Q & A

<<https://zainf.dev/fundamental-computer-science-ai-era>>

<<https://github.com/zainfathoni/agent-workflows>>

<<https://zainf.dev/a-z-3>>